SONDRA
Music Recognition System
Technical Design and Requirements Document

Emir Furkan Karahasanoğlu

March 30, 2026

Contents

1 Introduction

1.1 Purpose of the Document

The purpose of this document is to define the requirements, architectural structure, basic working principles, data flow between system components, and user interactions of the Sondra music recognition system in a formal and technical framework. This document serves as a common reference for the development team, establishes the boundaries within which the system will be developed, and explains how software components will interact with each other.

1.2 Project Definition

Sondra is a music recognition system that aims to identify a song by analyzing a short audio sample provided by the user through a mobile application. The system digitally processes the audio data, generates distinctive fingerprint data from it, compares the generated data with previously added song records in the system, and presents the result with the highest match to the user.

1.3 Scope

This document covers the following topics:

- System purpose and scope
- Basic concepts and definitions
- Functional and non-functional requirements
- General system architecture
- Communication between system components
- Audio processing and matching algorithm logic
- User interaction scenarios
- UML-based visual models
- Dataset creation process
- Song indexing and querying process using Barış Manço – Gülpembe as an example

1.4 Objectives

The main objectives of the Sondra system are listed below:

- Find the correct song from the short audio sample provided by the user
- Perform song recognition within a reasonable time
- Provide an easily accessible experience via mobile devices
- Allow new songs to be added to the system later
- Build a modular and extensible system infrastructure

1.5 Assumptions and Constraints

The following assumptions have been considered in the system design:

- The user has an internet connection.
- The user will grant microphone permission to the application.
- Recognizable songs in the system have been previously added to the database.
- The audio sample sent by the user is not completely silent.

The main constraints are as follows:

- The system may not correctly identify songs not present in the database.
- Excessive environmental noise may reduce accuracy.
- The initial version focuses on song recognition; humming recognition is outside the scope.

2 Definitions and Basic Concepts

2.1 Fingerprint

A fingerprint is a distinctive numerical representation derived from the frequency and time relationships of an audio recording. In the Sondra system, a song is represented not by a single fingerprint but by many small fingerprint sets.

2.2 Peak

A peak refers to frequency points with strong energy at a specific moment in the spectrogram. These peak points form the basic inputs of the matching algorithm.

2.3 Hash

A hash is short, comparable data generated from two peak points and the time difference between them. These data are used for fast searching in the database.

2.4 Offset

Offset is the difference between the hash time information obtained during a query and the hash time information found in the database. Matches concentrated around the same offset play a significant role in finding the correct song.

3 System Overview

The Sondra system consists of four basic components:

1. Mobile application
2. Backend API
3. Fingerprint generation service
4. Database layer

The mobile application manages user interaction. The backend API handles requests from the mobile application and coordinates the appropriate services. The fingerprint service is responsible for audio processing and fingerprint generation. The database stores song information and their associated fingerprint records.

3.1 General Architecture

The high-level architecture of the system is shown in Figure 1.

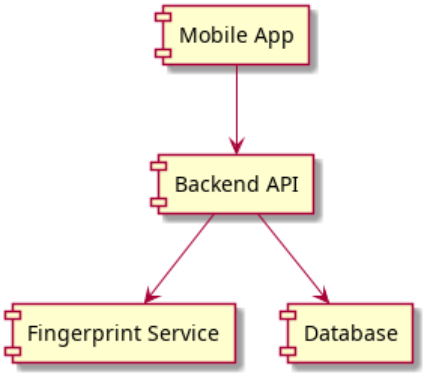

Figure 1: Sondra general system architecture

In this architecture, the mobile client does not communicate directly with the database; all communication is carried out through the backend API. Thus, security, authorization, error management, and business logic are controlled centrally.

4 User Interactions and Usage Scenarios

The primary user of the system is the end user using the mobile application. The user interacts directly with the system to start audio recording, send song recognition requests, and view results.

4.1 Use Case Diagram

The use case diagram showing the main usage scenarios of the system is given in Figure 2.

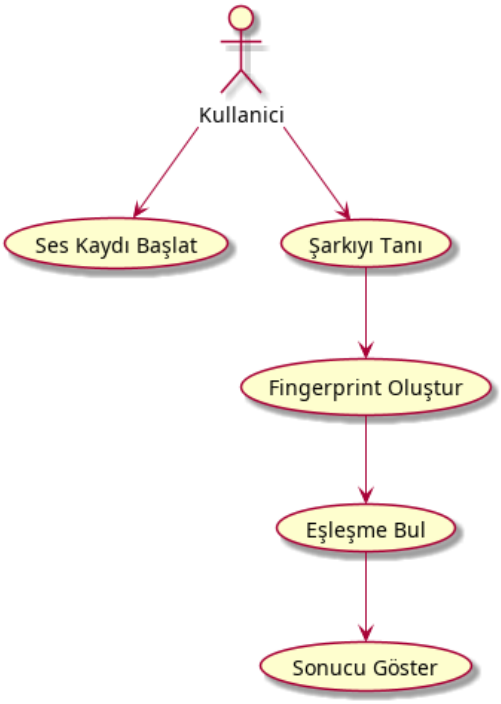

Figure 2: Sondra use case diagram

4.2 Basic Usage Scenario: Song Recognition

This scenario constitutes the most critical function of the system.

Actors: User

Preconditions:

- The user must have opened the application.
- Microphone access permission must have been granted.
- Internet connection must be active.

Basic Flow:

1. User enters the song recognition screen.
2. User presses the record button.
3. The application records ambient audio for a specific period.
4. Audio data is sent to the backend service.
5. The backend calls the relevant audio processing service.
6. The fingerprint service analyzes the audio data and generates hash values.
7. The backend queries the database for matches with these hash values.

8. The song with the highest and most consistent match is determined.
9. The result is transmitted to the mobile application.
10. The user sees the song name, artist name, and additional information on the screen.

Alternative Flows:

- If audio quality is too low, the system may not generate sufficient fingerprints.
- If no suitable match is found in the database, the user is informed that the song could not be identified.
- If a network error occurs, the user is notified to try again.

5 Functional Requirements

5.1 FR-1: Audio Recording

The system must be able to record audio from the user through the mobile device microphone. The recording duration should be configurable based on system parameters.

5.2 FR-2: Sending Audio Data to Server

The mobile application must send the acquired audio data to the backend API in the appropriate format. Connection errors should be manageable during data transmission.

5.3 FR-3: Fingerprint Generation

The system must process the incoming audio data, generate a spectrogram, detect peak points, and produce hash values from these points.

5.4 FR-4: Matching Search

The system must search the generated hash values in the database to identify potential song candidates.

5.5 FR-5: Result Calculation

The system must select the most appropriate song based not only on the number of hash matches but also on offset consistency.

5.6 FR-6: Result Display

The system must display the name and artist information of successfully identified songs to the user in an understandable format.

5.7 FR-7: New Song Addition

The system must be able to generate fingerprint data for new songs and add them to the database as part of the management or data preparation process.

5.8 FR-8: Failed Result Management

The system must display an appropriate information message to the user when no match is found.

6 Non-Functional Requirements

6.1 Performance

Under appropriate network and server conditions, the system should produce song recognition results in a short time. For the initial version, the target response time should be within a few seconds.

6.2 Scalability

System performance is expected to remain at acceptable levels as new songs are added to the database. The architecture should be extensible to accommodate larger datasets.

6.3 Accuracy

The system should focus on achieving high accuracy rates with clean or moderately noisy samples.

6.4 Usability

The mobile interface should be simple, understandable, and quickly accessible. Users should be able to initiate song recognition in as few steps as possible.

6.5 Maintainability

The system should be designed modularly; the mobile client, backend, audio processing service, and data layer should be separated from each other.

7 Technical Architecture and Components

7.1 Mobile Application

The mobile application is the main interaction layer between the user and the system. Its primary tasks include:

- Starting and stopping audio recording
- Sending recorded audio to the backend service
- Displaying results visually to the user
- Showing error and warning messages

7.2 Backend API

The backend API is the coordination layer of the system. Its main tasks include:

- Handling requests from the mobile client
- Calling the audio processing service
- Managing the matching process with the acquired hash information
- Sending results to the mobile application in a standardized format

7.3 Fingerprint Service

This service contains the audio processing logic. Basic steps:

1. Pre-processing of audio data
2. Frequency transformation
3. Spectrogram generation
4. Peak detection
5. Peak pairing
6. Hash generation

7.4 Database Layer

The database is the layer where song information and fingerprint records are stored. Due to the need for fast searching, a design suitable for hash-based querying should be adopted.

8 Data Flow and Operation

The operational sequence of the system is shown with a sequence diagram in Figure 3.

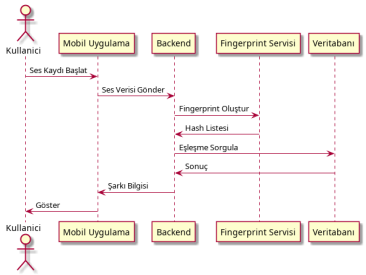

Figure 3: Sequence Diagram

In this flow, the user initiates audio recording through the mobile application. The mobile application sends the recording to the backend service. The backend calls the audio processing service and compares the obtained hash list with the database. The result is returned to the user when found.

9 Matching Algorithm Logic

In the Sondra system, matching is performed not through a single global signature, but through many small hash values obtained from an audio sample. Each hash represents two specific peak points and the time difference between them.

9.1 Algorithm Steps

1. A hash list is obtained from the audio sample.
2. Corresponding records for each hash are found in the database.
3. The difference between query time and database time is calculated for each match.
4. Matches clustered around the same song and similar offset are counted.
5. The song with the highest density and most consistent cluster is selected as the candidate.

9.2 Why Use Offset?

Looking only at the number of hash matches can lead to false positive results. However, since matches from the correct section of the same song cluster around a similar time shift, offset analysis significantly increases accuracy.

10 Data Model

The system data is expected to consist of two main logical structures: song information and fingerprint records.

10.1 Song Table

The song table may contain the following basic fields:

- Song ID
- Song name
- Artist name
- Album name
- Duration

10.2 Fingerprint Table

The fingerprint table may contain the following fields:

- Hash value
- Song ID
- Time information

This structure allows quick access to related song IDs and time points from a hash value.

11 Database Creation Process

This section explains how the music data to be used in the system will be collected, standardized, and processed. Dataset quality directly affects the system's accuracy rate and scalability.

11.1 Dataset Sources

The music dataset required for the system can be obtained from the following sources:

- Local music archives
- Open datasets (Free Music Archive, etc.)

- Audio from digital platforms
- Manual data collection processes

11.2 Database Creation Steps

The dataset creation process consists of the following basic steps:

1. Obtaining the songs to be used in the system
2. Converting audio files to a common format
3. Standardizing audio levels and sampling rates
4. Generating fingerprints for each song
5. Saving the obtained hash values to the database

11.3 Format and Standardization

For processing convenience and consistency, it is recommended to arrange audio files to have the same technical specifications as much as possible. In this context:

- WAV format should be preferred.
- A fixed sampling rate should be used.
- Noise reduction and basic pre-processing steps should be applied.

12 Song Indexing and Querying Process Using Barış Manço – Gülpembe as an Example

This section explains the system's working logic using Barış Manço's song *Gülpembe* as an example. Thus, both the database recording phase and the user query and matching process are demonstrated concretely.

12.1 Song Indexing

When adding a song of approximately 3 minutes (180 seconds) to the system, the audio is not stored as a single piece but divided into small analysis windows. This is necessary to monitor frequency content over time.

- Song duration: 180 seconds
- Window size: 64 ms – 128 ms

- Total number of windows: approximately 3000 – 5000

A Fourier transform is applied to each window. From the resulting spectrogram data, only high-energy frequency points are selected. In other words, the system does not store all frequency information for each moment; instead, it uses more distinctive peak points.

For example:

- A strong peak around 500 Hz at 10 seconds
- A second peak around 1200 Hz at 10.2 seconds

These points are then evaluated as pairs, and a hash value is generated from each pairing.

12.2 Hash Generation Density

In an average configuration, it can be assumed that approximately 30 – 50 hashes are generated per second. In this case:

$$180 \times 40 \approx 7200$$

approximately 7200 hash values are obtained. This shows that a single song is represented by thousands of small fingerprints in the database.

12.3 Database Record Structure

Hash values belonging to the song Gülpembe are stored with the following logic:

Table 1: Sample Fingerprint Records for the Song Gülpembe

Hash Value	**Song ID**	**Time (s)**
88231	001 (Gülpembe)	0.5
11294	001 (Gülpembe)	0.8
24567	001 (Gülpembe)	15.4
38901	001 (Gülpembe)	78.7
55902	001 (Gülpembe)	179.2

Thanks to this structure, time information for every part of the song, whether beginning, middle, or end, is stored. This means that even if the user listens to any part of the song, the system can make matches using the hash sets belonging to that section.

12.4 Query Phase

When the user sends a 5 – 10 second recording through the application, the same fingerprint generation process is applied to this short sample. For example, suppose the user obtains two hashes as follows:

- Hash 55092 at 1.0 seconds of the user's recording
- Hash 12883 at 1.5 seconds of the user's recording

These hash values are searched in the database. Each hash value can be found not only in one song but also in different songs. However, the determining factor is the temporal pattern in which these hashes are found.

12.5 Offset Calculation

The system calculates the following difference for each match:

$$\Delta T = T_{song} - T_{query}$$

For example, suppose the following results are obtained for Gülpembe:

Table 2: Offset Calculation Example for Gülpembe

Hash	T_{song}	T_{query}	ΔT
55092	45.0	1.0	44.0
12883	45.5	1.5	44.0
99201	46.1	2.1	44.0

The difference being constant around 44.0 for all matches indicates that the sample sent by the user corresponds to the section around the 45th second of the song.

12.6 Decision Mechanism

In incorrect candidate songs, the same hash values will be found at random times, so offset values will be scattered. In contrast, many hashes for the correct song will cluster around the same time difference. Therefore, the system looks not only at which hashes match but also at whether the temporal relationships of these hashes are consistent.

Thanks to this mechanism:

- If the user listens to the beginning of the song, hash sets belonging to the first seconds match.
- If the user listens to the middle of the song, hash sets belonging to the middle section match.

- If the user listens to the end of the song, hash sets belonging to the final sections match.

Thus, the system is not dependent on a single fixed point; since the entire song is indexed, every section can be recognized.

12.7 Key Interpretation

This example shows that the Sondra system answers the question "In which song do these sounds appear together with their time distances preserved?" rather than "Does this sound exist in this song?" This is precisely why the system can work with high accuracy even in noisy environments.

13 Error Conditions and Exceptions

The system should handle the following error conditions:

- Microphone permission being denied
- No network connection
- Server-side service errors
- Insufficient audio quality
- No match found in the database

For each error condition, the user should be presented with a message that does not contain technical details, is understandable, and provides guidance.

14 Future Extensibility Options

The following features can be added to the Sondra system in future versions:

- Humming-based song recognition
- Limited offline recognition support
- Song history viewing
- Adding to favorites
- Song preview links
- Admin panel for batch data upload

15 Conclusion

Sondra is a modular music recognition system consisting of a mobile client, backend coordination layer, audio processing service, and database components. The system generates distinctive fingerprint data from short audio samples and compares these with previously prepared records to determine the most appropriate result. This document has detailed the system's purpose, boundaries, basic requirements, data flow, technical structure, dataset creation process, and indexing and querying mechanism using the Barış Manço – Gülpembe example. The prepared structure provides a design framework that can serve as a basis for both academic project reporting and actual development processes.